

MSDM 5001 - Computation and Simulation Tools

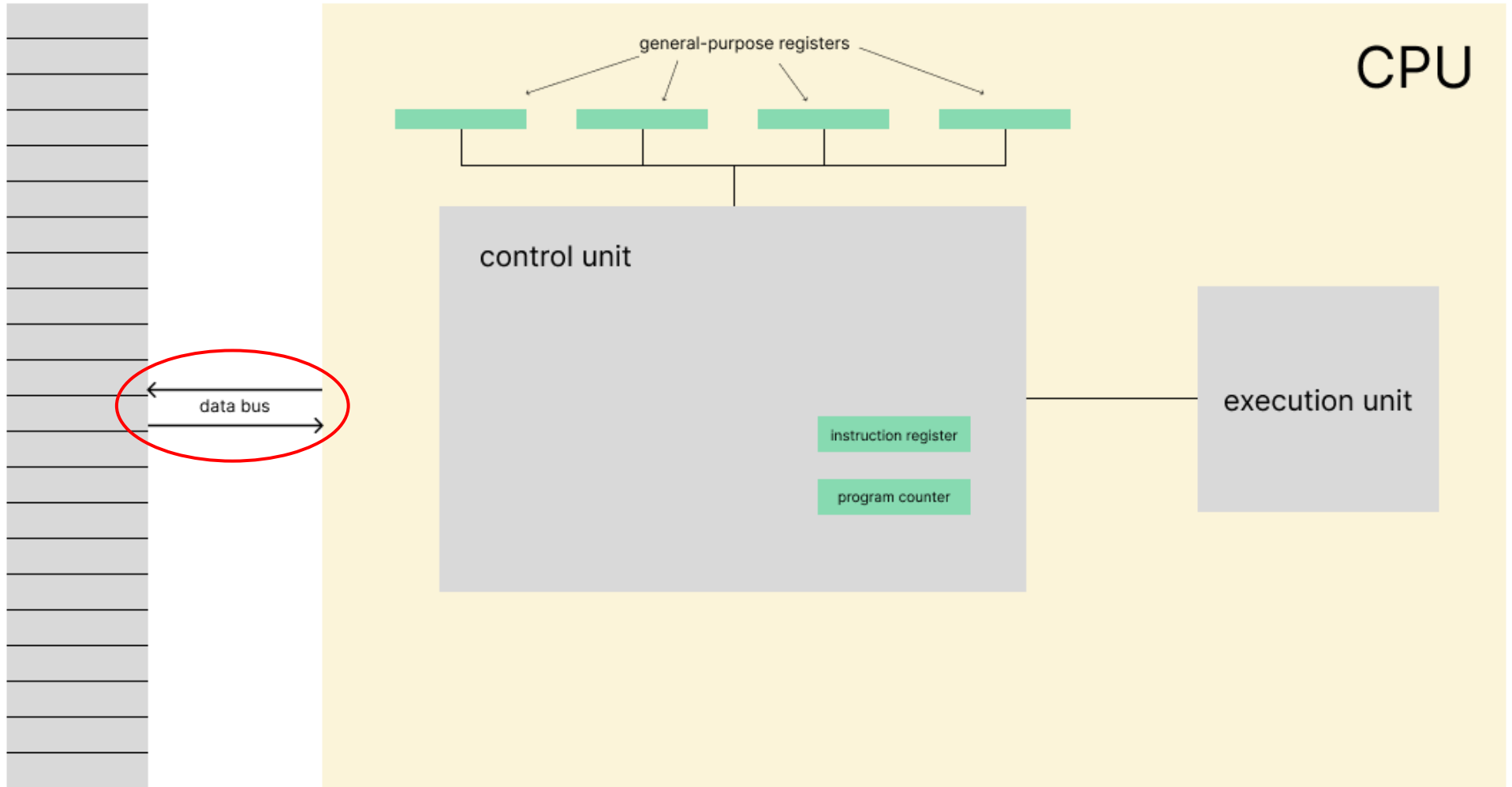
Lecture 4: Basic computer architecture, CPUs and GPUs

von-Neumann architecture (1945)

- CPU + memory
- Data and instructions on equal footing
- Fetch, decode, execute cycles
 - Fetch: CPU loads instruction into the memory (in binary)
 - Decode: CPU interprets the instruction
 - Execute: Execution unit carries out the interpreted instruction



main memory



An aside: Instruction Set Architecture

- Instruction set:
An interface that defines what operations a CPU can carry out
- Instructions are mapped to “OP codes” in binary
 - A hypothetical example:

100001010010

ADD Location of “x” Location of “y”

- x86 and ARM: different instruction sets
- Question: can a program compiled to run on x86 be run on ARM?
- Try finding out your CPU’s architecture by using `lscpu`

Program = sequence of instructions

```
1 int main() {  
2     int x = 5, y = 3;  
3     int z = x + y;  
4     return z;  
5 }
```



```
1 main:  
2     push    rbp  
3     mov     rbp, rsp  
4     mov     DWORD PTR [rbp-4], 5  
5     mov     DWORD PTR [rbp-8], 3  
6     mov     edx, DWORD PTR [rbp-4]  
7     mov     eax, DWORD PTR [rbp-8]  
8     add     eax, edx  
9     mov     DWORD PTR [rbp-12], eax  
10    mov     eax, DWORD PTR [rbp-12]  
11    pop     rbp  
12    ret
```

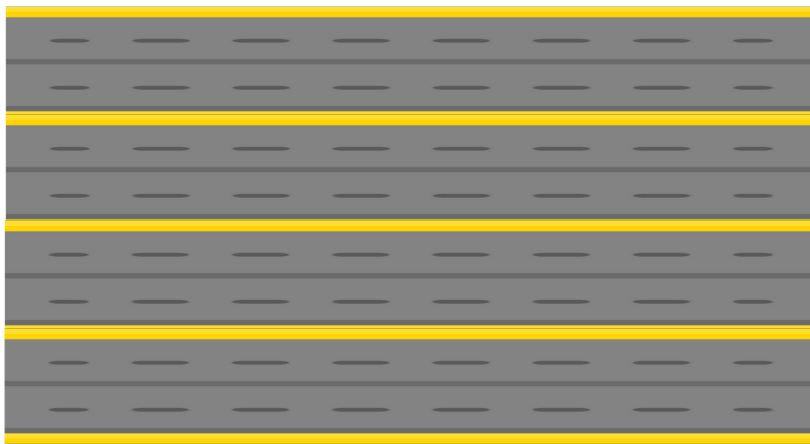
“Threads”

- In this lecture, “thread” means
 - One stream of instructions, **AND**
 - The state of the CPU while executing those instructions
(i.e. values store in the program counters and registers and relevant memory locations)
- Different threads may be executing the SAME program!
(Threads are “dynamic”, programs are “static”)

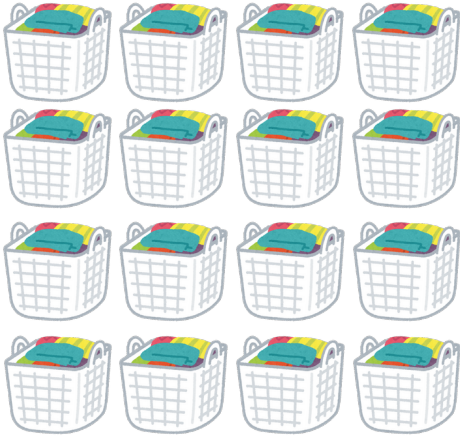


The von-Neumann Bottleneck

- CPU speed has increased a lot over the years (kHz $\rightarrow$ MHz $\rightarrow$ GHz)
- Memory “speed”: not so much...
 - Latency: How long it takes to fetch data from memory
 - Bandwidth: How much data can be sent per unit time (after initial latency)



An illustrative example



45 mins



60 mins



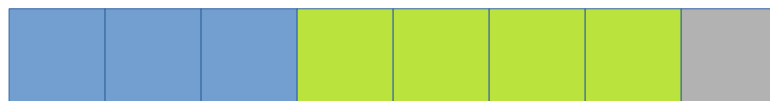
15 mins

What is the latency of doing 1 load of laundry?
What if I have two washers and two dryers?

An illustrative example

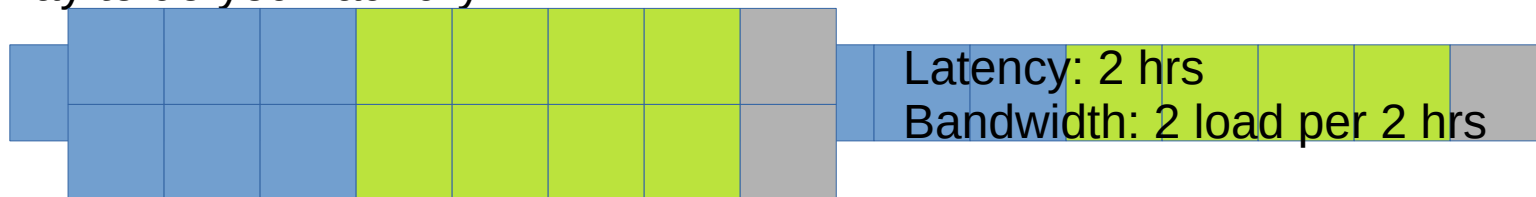


1 washer + 1 dryer



Latency: 2 hrs
Bandwidth: 1 load per 2 hrs

2 washers + 2 dryers
One way to do your laundry...

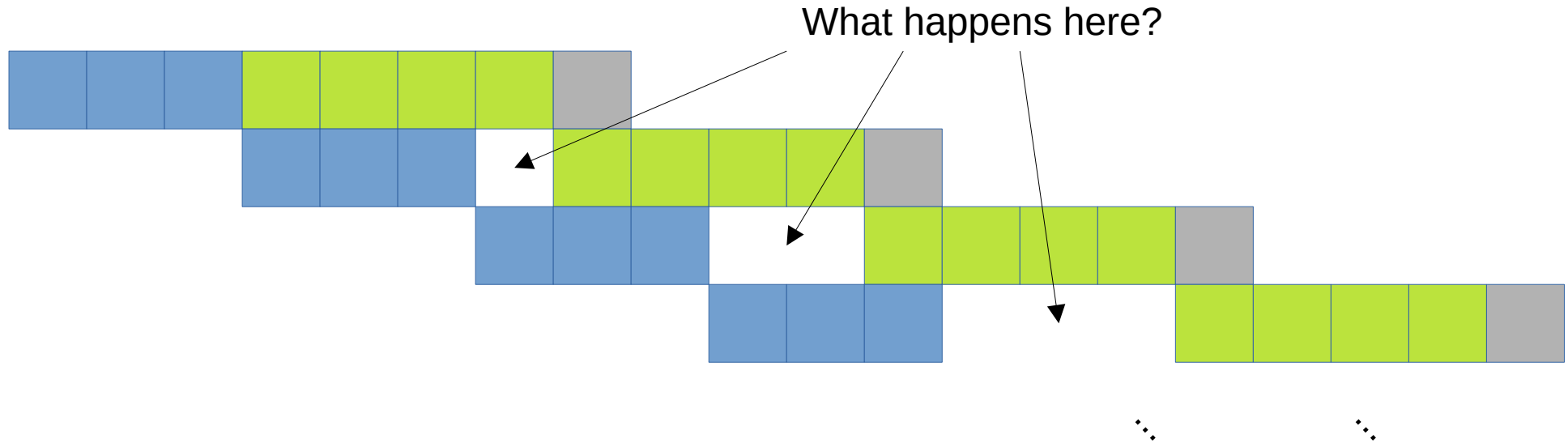


Latency: 2 hrs
Bandwidth: 2 load per 2 hrs

Is this the best use of resources?

1 washer + 1 dryer

An illustrative example



What is the throughput (loads of laundries done per unit time)?

Has bandwidth changed? Has latency changed?

What are the utilization ratios of the washer, dryer and you?

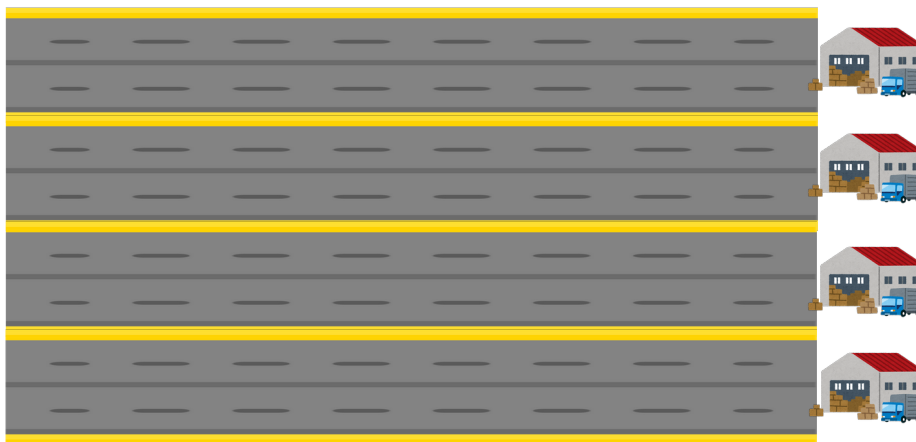
How would answers to questions above change if I have 1 washer and 2 dryers?

Caches

Caches (sum of all):

L1d:	256 KiB (8 instances)
L1i:	256 KiB (8 instances)
L2:	4 MiB (8 instances)
L3:	16 MiB (1 instance)

- Memory directly built on chip – very fast!
- Much smaller in size than main memory (why??)
- Loads data in “batches” – cache lines
 - Principle of spatial and temporal locality



Caches

Main
memory

0x0	2
0x1	7
0x2	-1
0x3	0
0x4	9
0x5	3
0x6	5
0x7	5
0x8	2
0x9	0
0xA	0
0xB	0

CPU

Caches

Line address	values
0x2	-1, 0, 9, 3
0x7	5, 2, 0, 0

cache hits
vs
cache misses

2 cache lines, 16 bytes each

Caches

Assume:

- 2 cache lines, 16 bytes each
- LRU policy

0x0	2
0x1	7
0x2	-1
0x3	0
0x4	9
0x5	3
0x6	5
0x7	5
0x8	2
0x9	12
0xA	-5
0xB	-7
0xC	0
0xD	0
0xE	1
0xF	4

Operation	Results	Caches
LOAD 0x0	Cache miss	Line1: 2, 7, -1, 0 Line2:
LOAD 0x1	Cache hit	Line1: 2, 7, -1, 0 Line2:
LOAD 0xA	Cache miss	Line1: 2, 7, -1, 0 Line2: -5, -7, 0, 0
LOAD 0xD	Cache hit	Line1: 2, 7, -1, 0 Line2: -5, -7, 0, 0
LOAD 0x5	Cache miss, evict line1	Line1: 3, 5, 5, 2 Line2: -5, -7, 0, 0
WRITE 0x0	?	?

Some numbers

Table 1: CPU Operation Timing

Operation	Typical Time	Typical Size
Adding two integers	0.5 ns	—
Dividing two integers	20 ns	—
Accessing registers	0.5 ns	32-64bits per register
Accessing L1 cache	1-2 ns	32-64KB
Accessing L2 cache	3-5 ns	256KB - 1MB
Accessing L3 cache	10-20 ns	8-32MB
Accessing main memory	100-300 ns	8-128GB
Reading from SSD	$\mathcal{O}(10^4)$ ns	128-4096 GB

Demo: Effects of caches

```
5  const int ROWS = 1000;
6  const int COLS = 1000;
7
8  int matrix[ROWS][COLS];
9
10 void initializeArray() {
11     for (int i = 0; i < ROWS; i++) {
12         for (int j = 0; j < COLS; j++) {
13             matrix[i][j] = i * COLS + j + 1;
14         }
15     }
16 }
17
18 long long sumRowFirst() {
19     long long sum = 0;
20     for (int i = 0; i < ROWS; i++) {
21         for (int j = 0; j < COLS; j++) {
22             sum += matrix[i][j];
23         }
24     }
25     return sum;
26 }
27
28 long long sumColumnFirst() {
29     long long sum = 0;
30     for (int j = 0; j < COLS; j++) {
31         for (int i = 0; i < ROWS; i++) {
32             sum += matrix[i][j];
33         }
34     }
35     return sum;
36 }
```

```
• (.eric) eric@fedora:~$ ./cache_effects time
Row-first sum: 500000500000 (Time: 1091 µs)
Column-first sum: 500000500000 (Time: 2157 µs)
Speedup ratio: 1.97709x
```

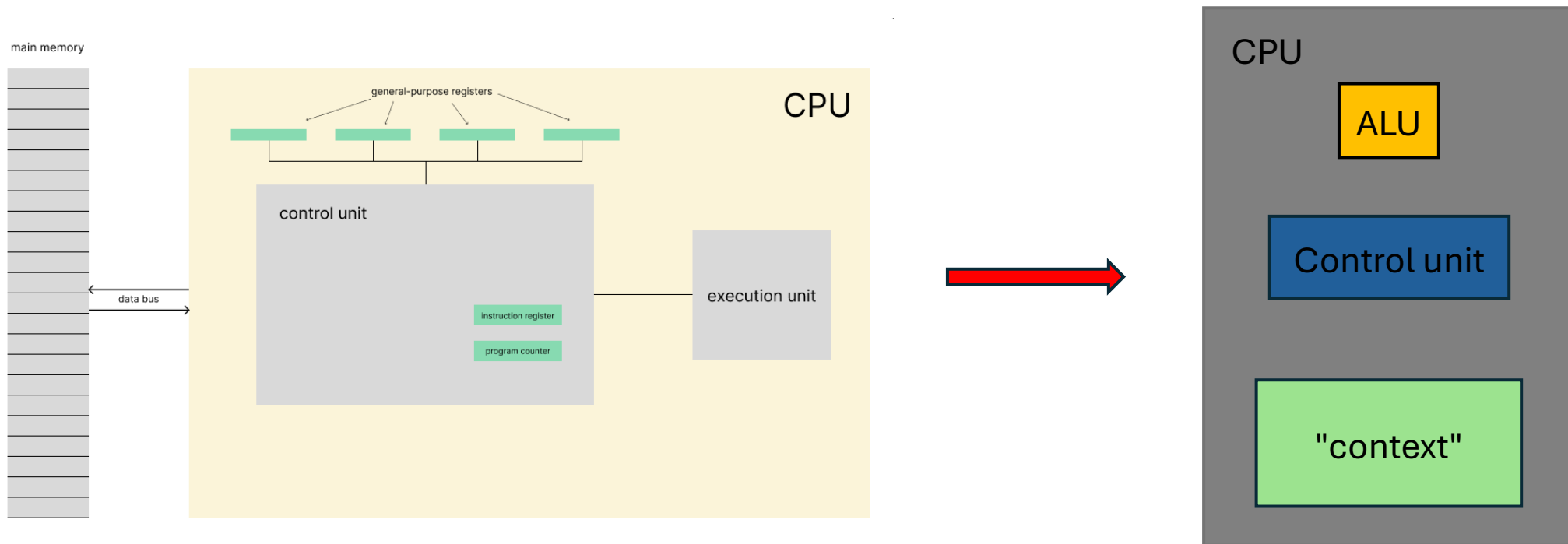
Performance counter stats for './cache_effects row':

270,684	cache-references:u	
20,144	cache-misses:u	# 7.44% of all cache refs
29,639,427	instructions:u	# 2.82 insn per cycle
10,505,239	cycles:u	

Performance counter stats for './cache_effects col':

1,340,479	cache-references:u	
212,281	cache-misses:u	# 15.84% of all cache refs
29,639,605	instructions:u	# 2.45 insn per cycle
12,078,967	cycles:u	

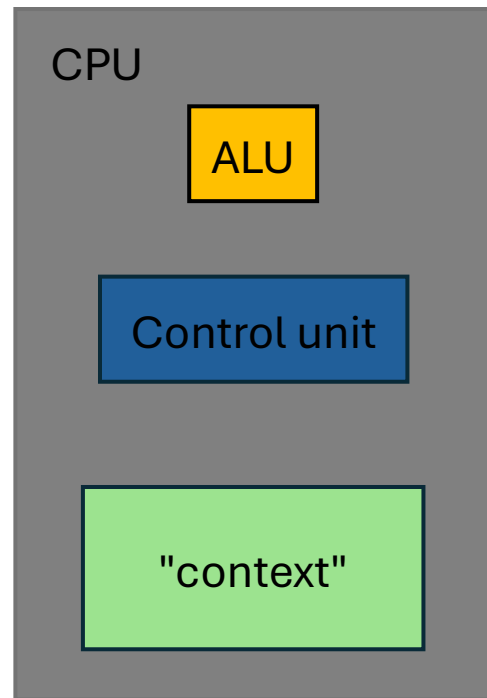
Let's simplify our representation of a CPU (ignoring memory):



Goal: modify so that it can run "faster"

Pipelining

- Interleaving different stages of operations
 - Fetch: memory management unit
 - Decode: control unit
 - Execute: execution unit
- Modern CPUs have much longer pipelines
e.g. Intel i5 chips have ~14 stages of pipelines
- Note: independent of programs
- “Latency hiding”



Branch prediction

- How to pipeline when have an if-else branch??
- CPUs are very good at guessing which branch the program will take (reportedly 90+% accuracy!)
- Penalty of wrong prediction also increases with longer pipeline...
 - Need to flush whole pipeline if prediction gets wrong

Out-of-order execution

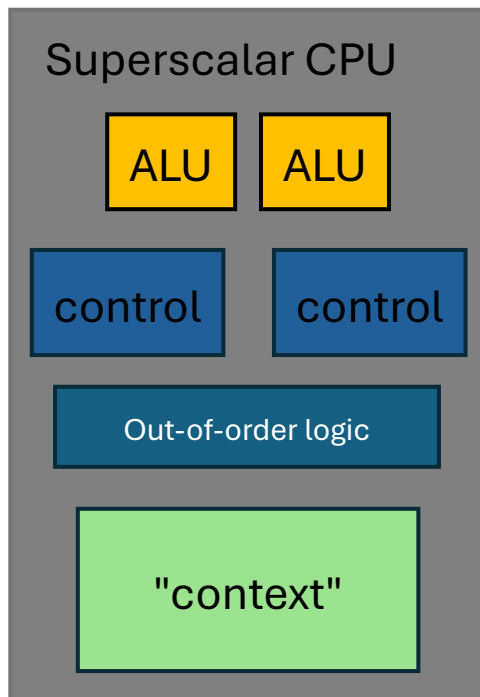
- A (pseudo-assembly) program:

```
LOAD    R0    0x1001  ← Slow!  
ADD     R0    R1  
MUL     R6    R7
```

- Observe: line 3 independent of line 1
- Immediately execute line 3 after line 1, skipping line 2!
- Instruction-level latency hiding (program dependent)

Instruction-level parallelism (ILP)

```
int x = a + b;  
int y = c * 2;  
int z = x + y;
```



Superscalarity:

Ability to execute independent instructions simultaneously in one thread

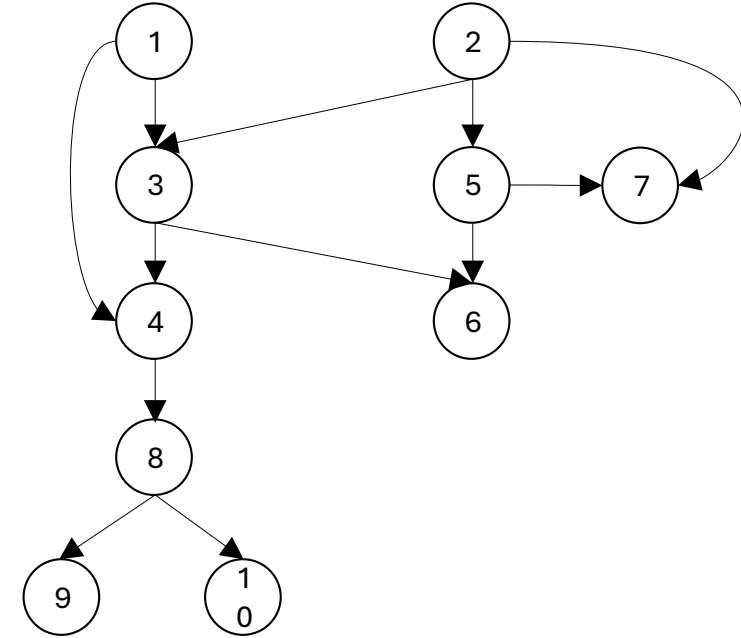
Diminishing returns!

An example of instructions dependency

```
1  a=1
2  b=2

3  c=a+b
4  d=a+c
5  e=2*b
6  f=c+e
7  g=e+b

8  if d>3:
9      print("Hello")
   else:
10     print("World")
```

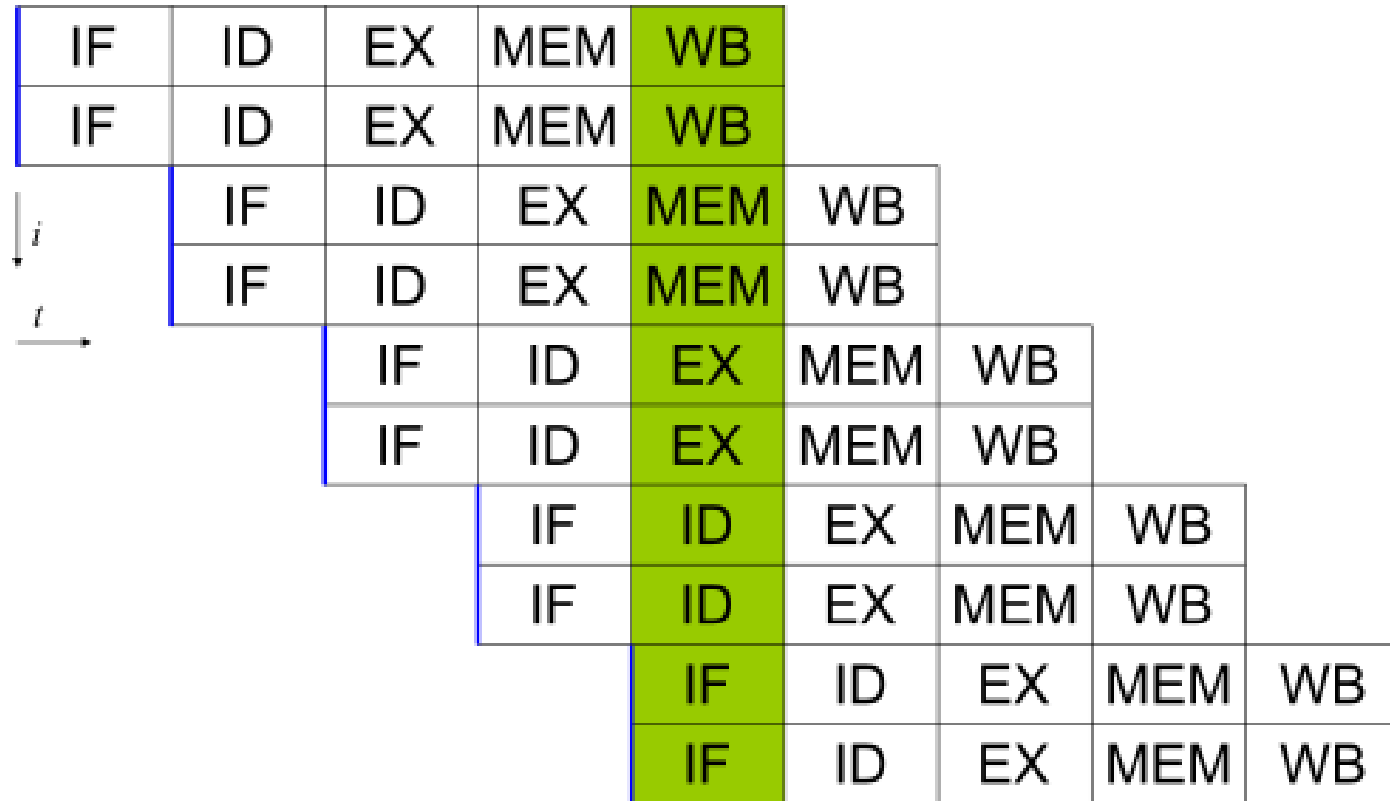


Task: Draw the dependency graph for this program.

Assume each line of code only takes 1 clock cycle to execute,
what is the minimum number of clock cycles needed?

How many execution units do we need?

Superscalarity + Pipelining



SIMD: Single Instruction, Multiple Data

- Consider the following program:

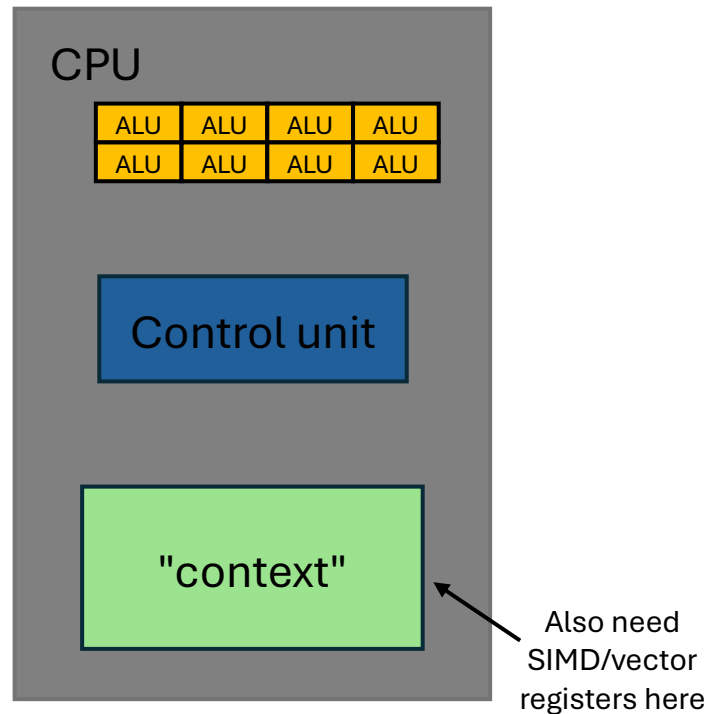
```
# x, y, z initialized here

for i in range(1_000_000):
    z[i] = x[i] + y[i]
```

- “Data parallelism”
- Can we rely on superscalarity in situations like this?

SIMD: Single Instruction, Multiple Data

- Programmers specify what sections of code can be executed in parallel
- Separate instruction sets
 - avx, avx512, sse, ...
 - Hypothetical example: LOAD → VLOAD
- Can be specified via compiler intrinsics, or using special libraries
- In numpy, **some** functions are compiled to use SIMD (if available)



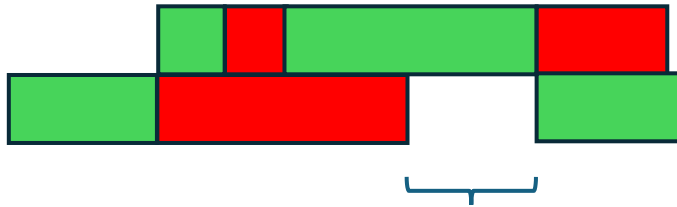
Simultaneous Multithreading (SMT)/ Hyperthreading

- So far, we have focused on executing one stream of instructions faster
- What if we DO have to wait? (e.g. memory access)
- An obvious idea: do something else!
- Any potential issues?



Simultaneous Multithreading (SMT)/ Hyperthreading

time



Trade-off:
Latency of one thread increases, but
overall throughput is increased

Two instruction
streams share
same resources!

CPU

ALU

Control unit

context

context

Question: more threads = better?

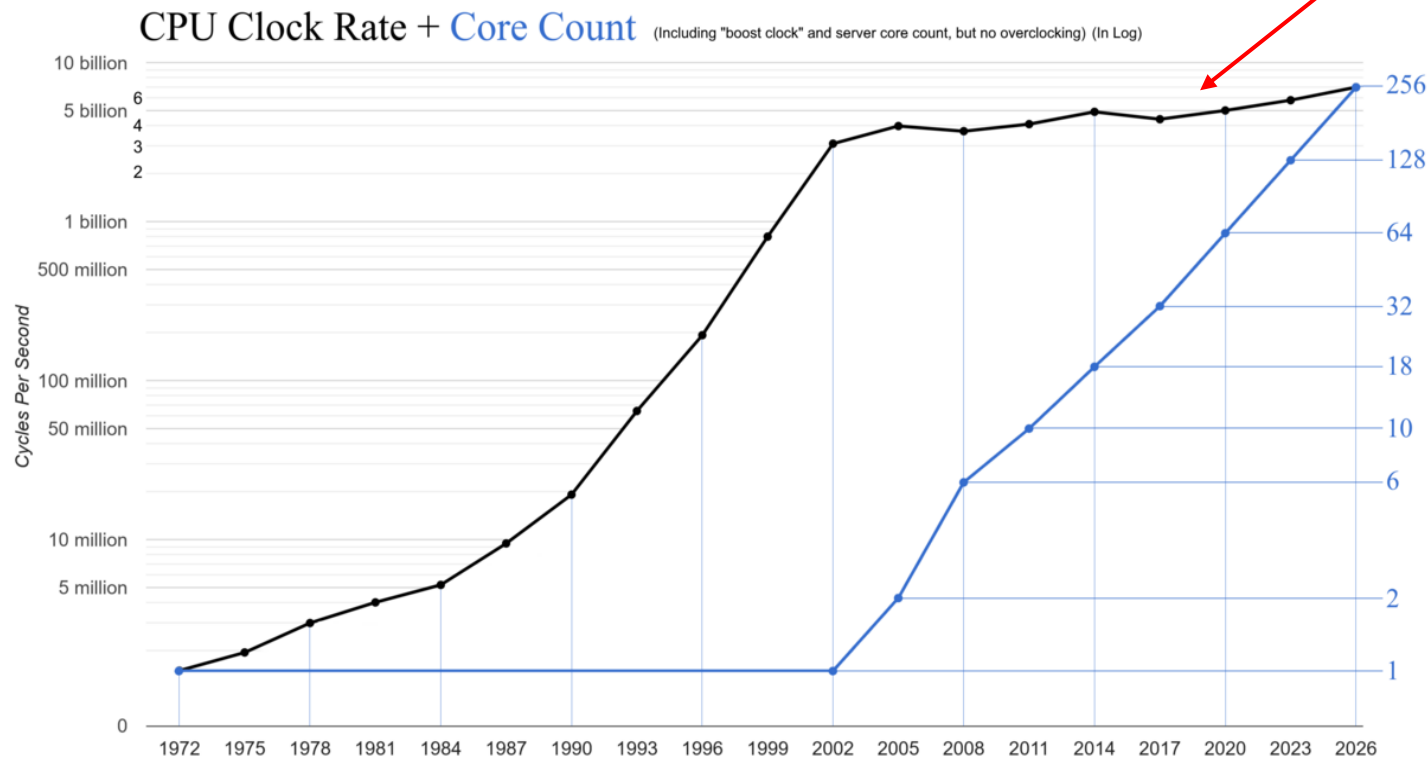
Ideas so far

- To make a single instruction stream faster:
 - Superscalarity
 - Out-of-order execution
 - SIMD
 - SMT
- Latency hiding

All on a single "core"!

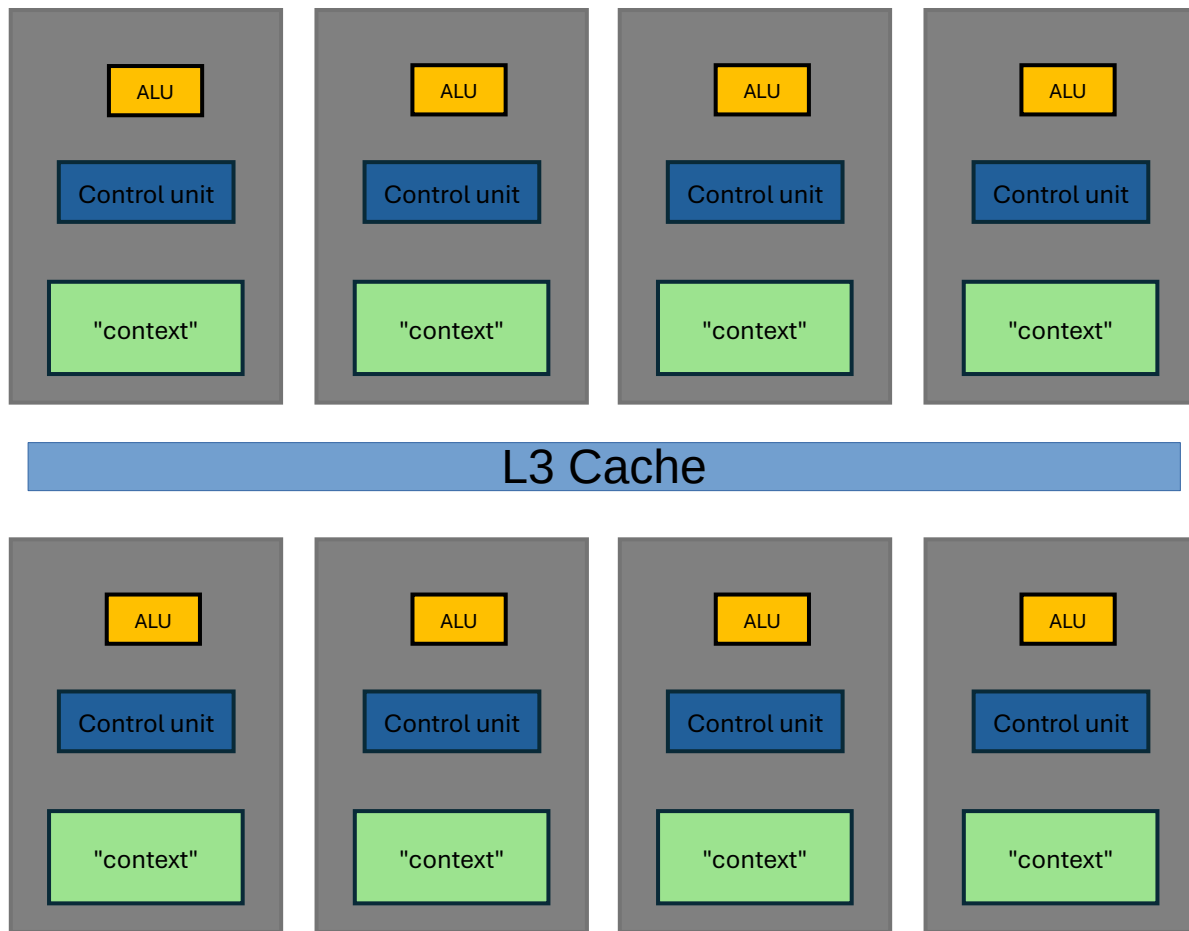
Rise of multi-core architecture

Why?

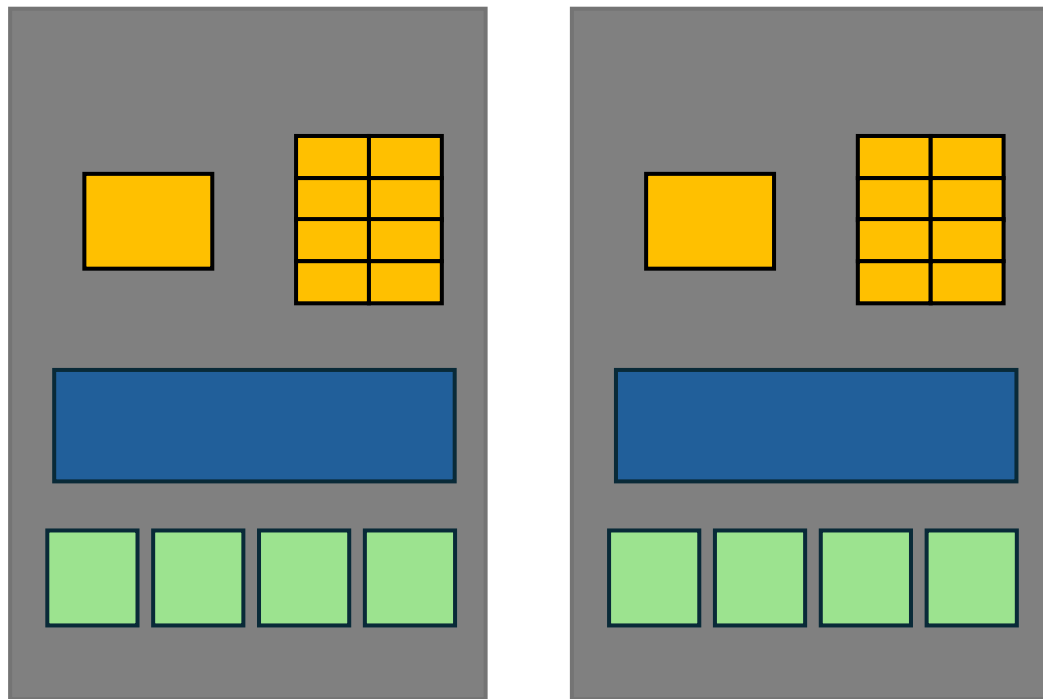


Source: <https://www.singularity.com/charts/page61.html> 1976- 1998 "Microprocessor clockspeed" (modified) | 1999-2026 Manufacturer specifications
Method: Fastest / heighest (not coupled) core count CPU released (1999-2024) 2026 - Leaks + conservative prediction

An example of a multi-core chip



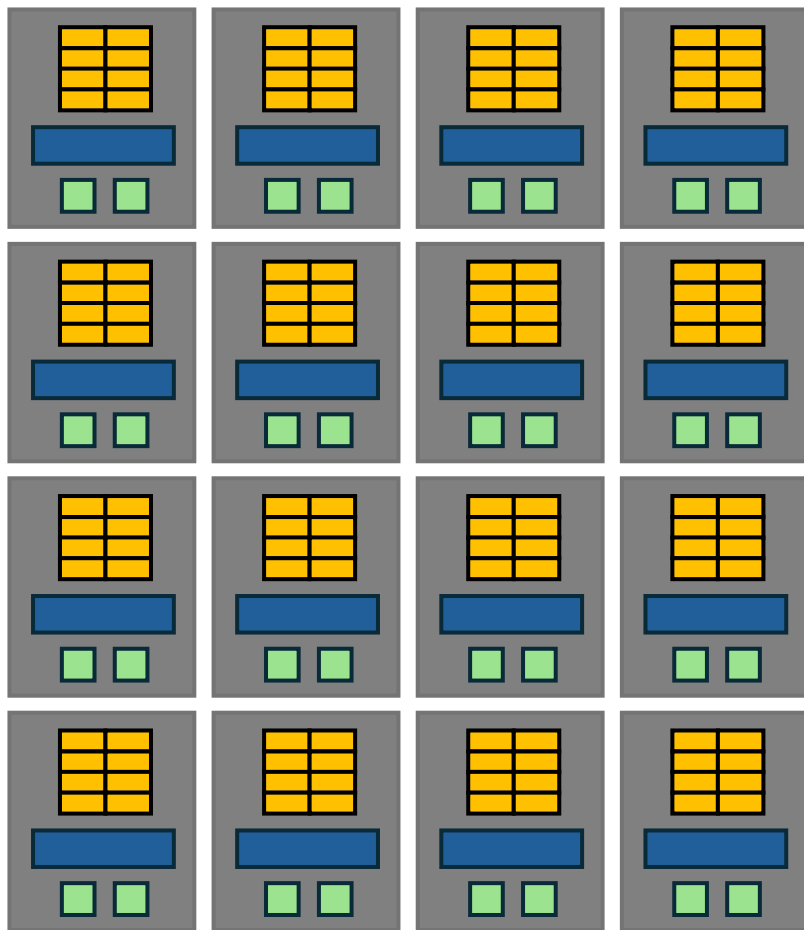
Mix and match...



Each core can execute up to 2 instructions per clock (1 scalar and 1 SIMD instructions), where instructions are from any of the four threads (implementation-dependent)

Dual-core, Multi-threaded (4-way), superscalar

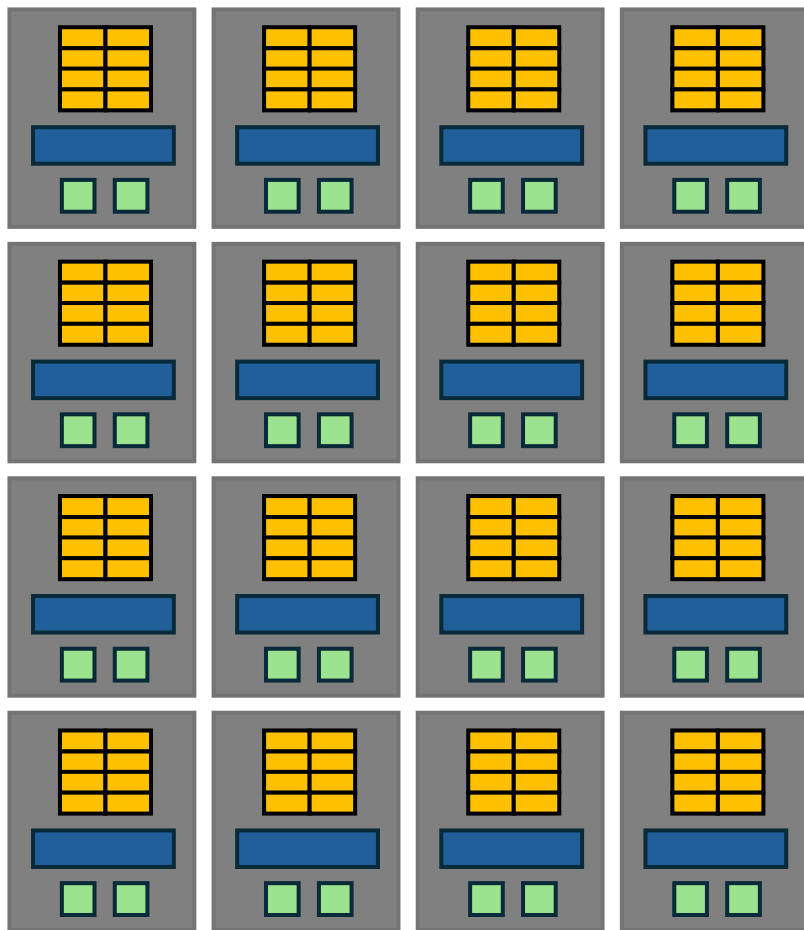
A hypothetical chip



Questions:

- 1) How many FLOP can be executed per clock?
- 2) How many threads can be executed at the same time?
- 3) How many threads can the chip “hold” at the same time?
- 4) How much “work” is needed to saturate this chip?

von-Neumann bottleneck & multi-cores



Assume:

Clock rate ~4 GHz

Latency ~100 ns

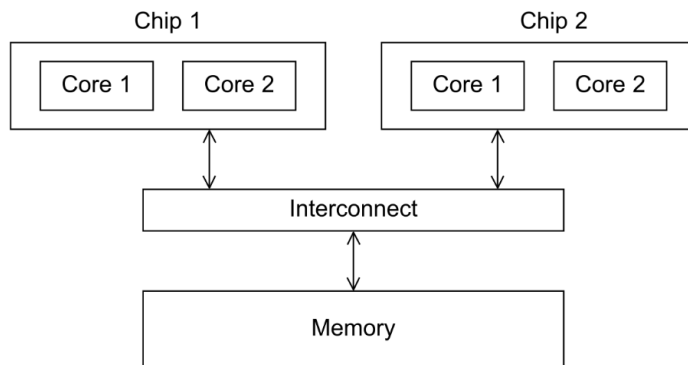
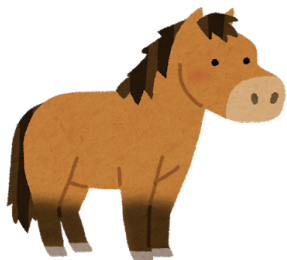
Bandwidth ~32 GB/s

If I want to use this chip to perform a vector addition of 32-bit floats, is it a “good” idea? Does caching help?

**In a lot of applications,
bandwidth is the limiting factor!**

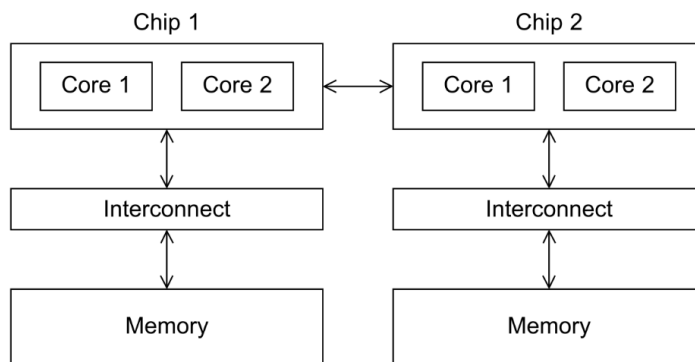
Memory systems in multi-core chips

- Uniform memory access (UMA)
 - Different cores have same latency and bandwidth to memory
 - Common in laptops and lower-end desktops/workstations (with ≤ 16 cores)
 - Advantage: easier to program
 - Disadvantage: _____



Memory systems in multi-core chips

- Nonuniform memory access (NUMA)
 - Different cores have different latencies and bandwidths to memory
 - Common in higher end servers/workstations with ≥ 16 cores
 - Advantage: _____
 - Disadvantage: more difficult to program



From last lecture...

- Why modern chips have caches, latency vs bandwidth
- Various ways to make a single core CPU run faster
 - Pipelining
 - Superscalarity
 - SIMD
 - SMT
- Why multi-core
- Multi-core architecture & related memory systems

Caches in multi-core chips

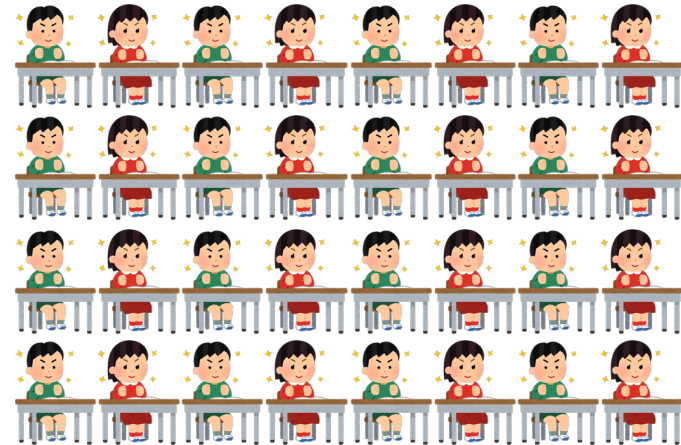
- What happens when different cores need to access same memory?
 - Core 1: Loads the data into its own private cache
 - Core 2: Loads the same data into its own private cache
 - Core 1: Modifies the data
 - Core 2: ???
- “Cache coherence”
 - Handled by hardware, automatic
 - Not free!

Caches in multi-core chips

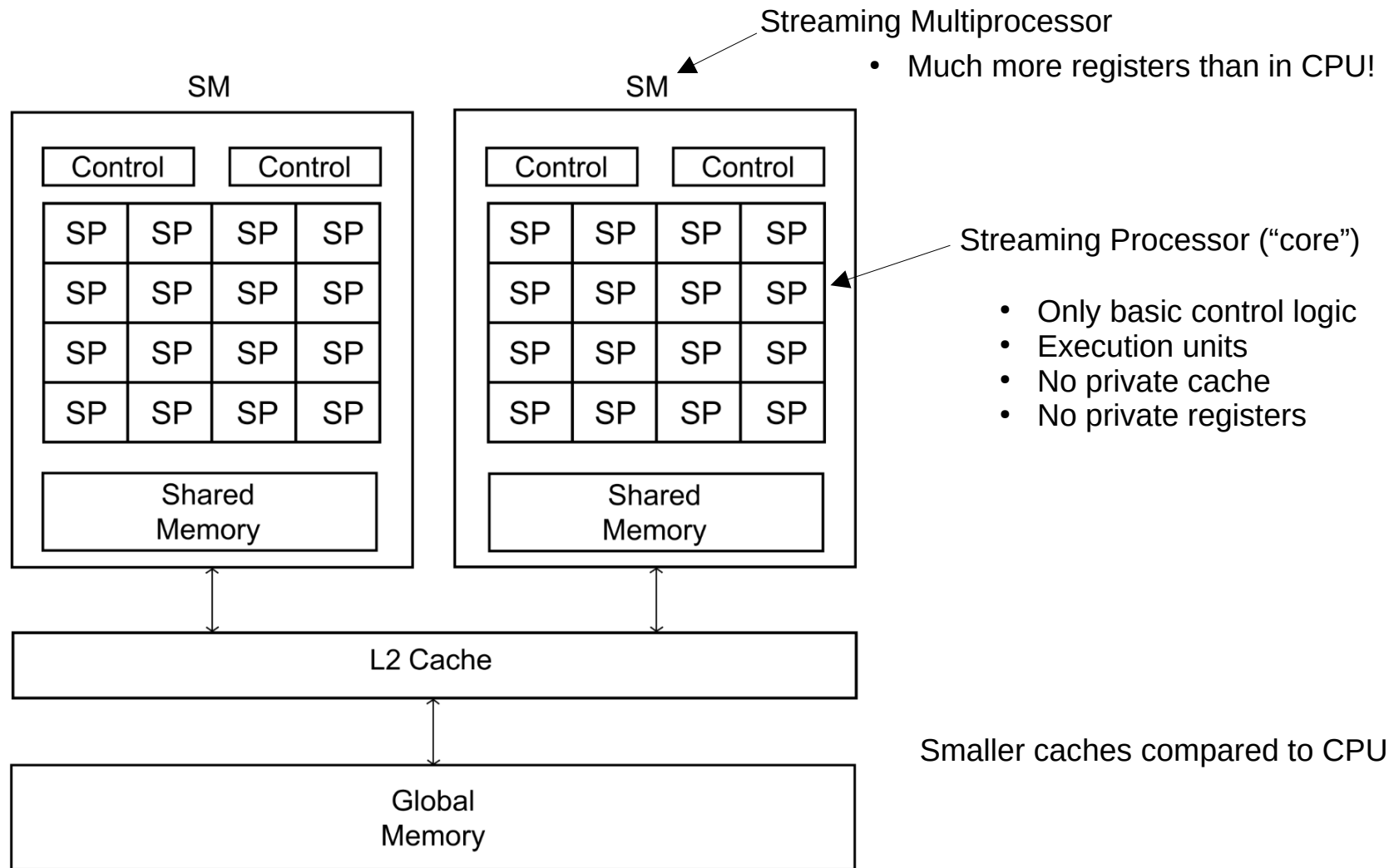
- False sharing
 - Core 1 needs to write to first 4 bytes of a cache line
 - Core 2 needs to write to the next 4 bytes
 - What would happen?
- See demo

GPU

- At a very high-level:
 - A multi-core CPU: A few very smart brains
 - GPU: A massive number of less sophisticated brains, SIMD on steroids

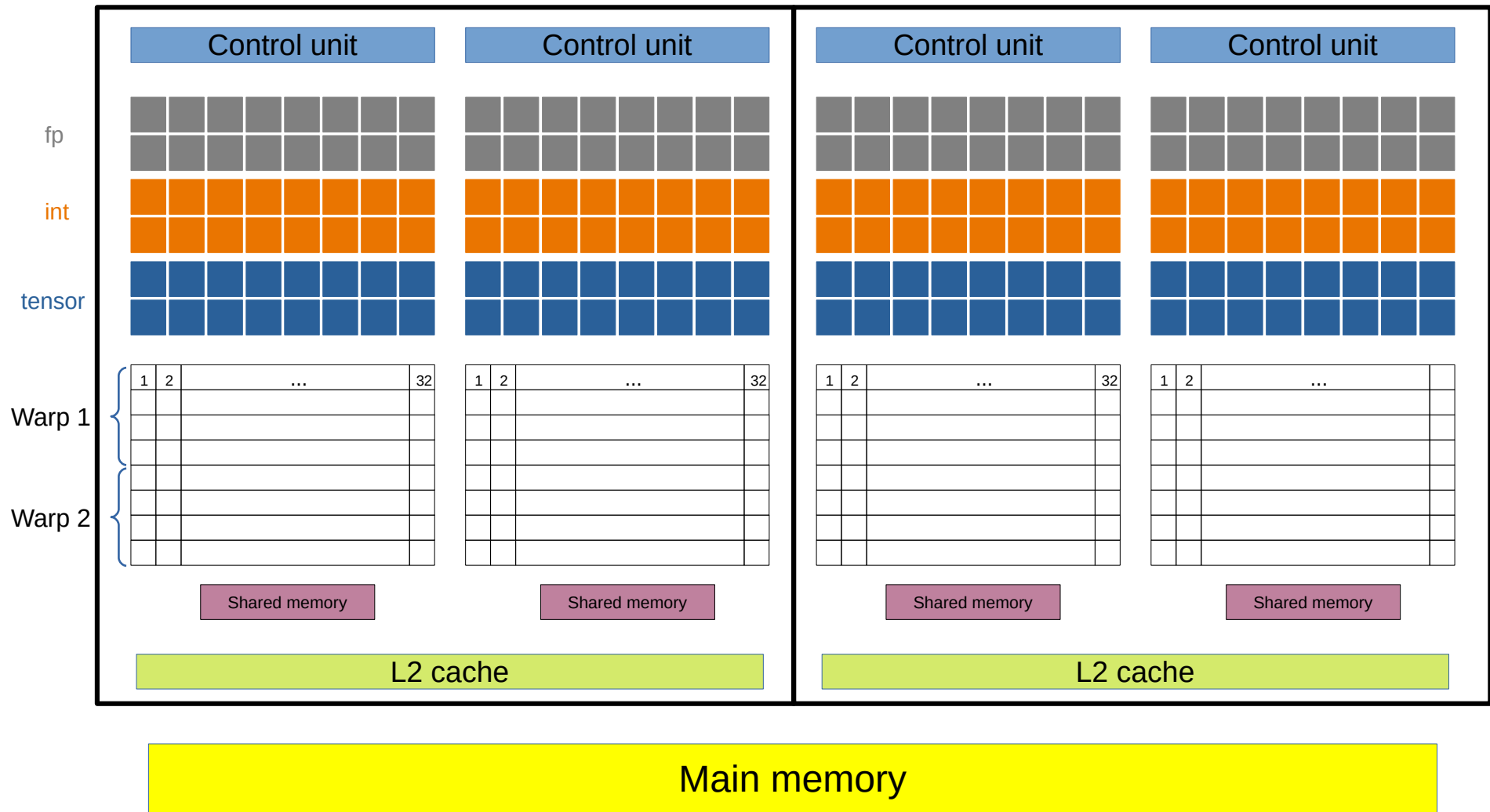


GPU

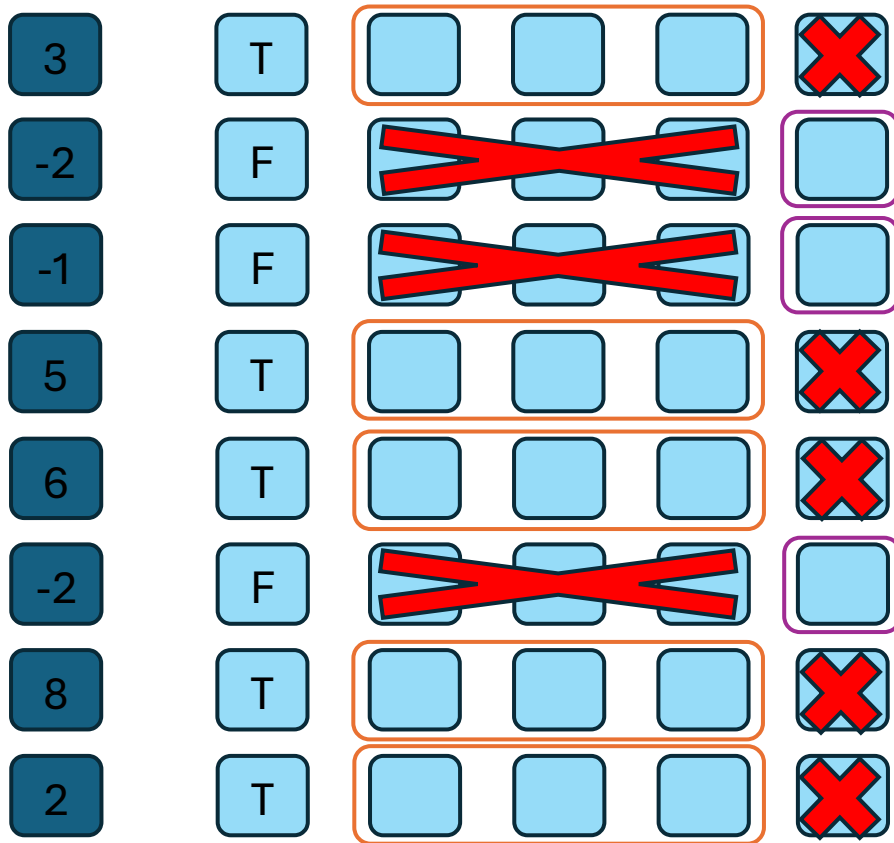


GPU Hierarchy

- Warp
 - 32 threads all executing the same instructions (SIMT vs SIMD)
 - “Effectively” a 32-way SIMD
- Thread block
 - A group of warps, number specified by programmer
 - Executed in the same SM
 - Share the same register files on the SM
 - Multiple warps can be in-flight at the same time! When one warp stalls, immediately switch to another. (c.f. SMT)
- Grid
 - A group of thread blocks, defined by programmer



Branching



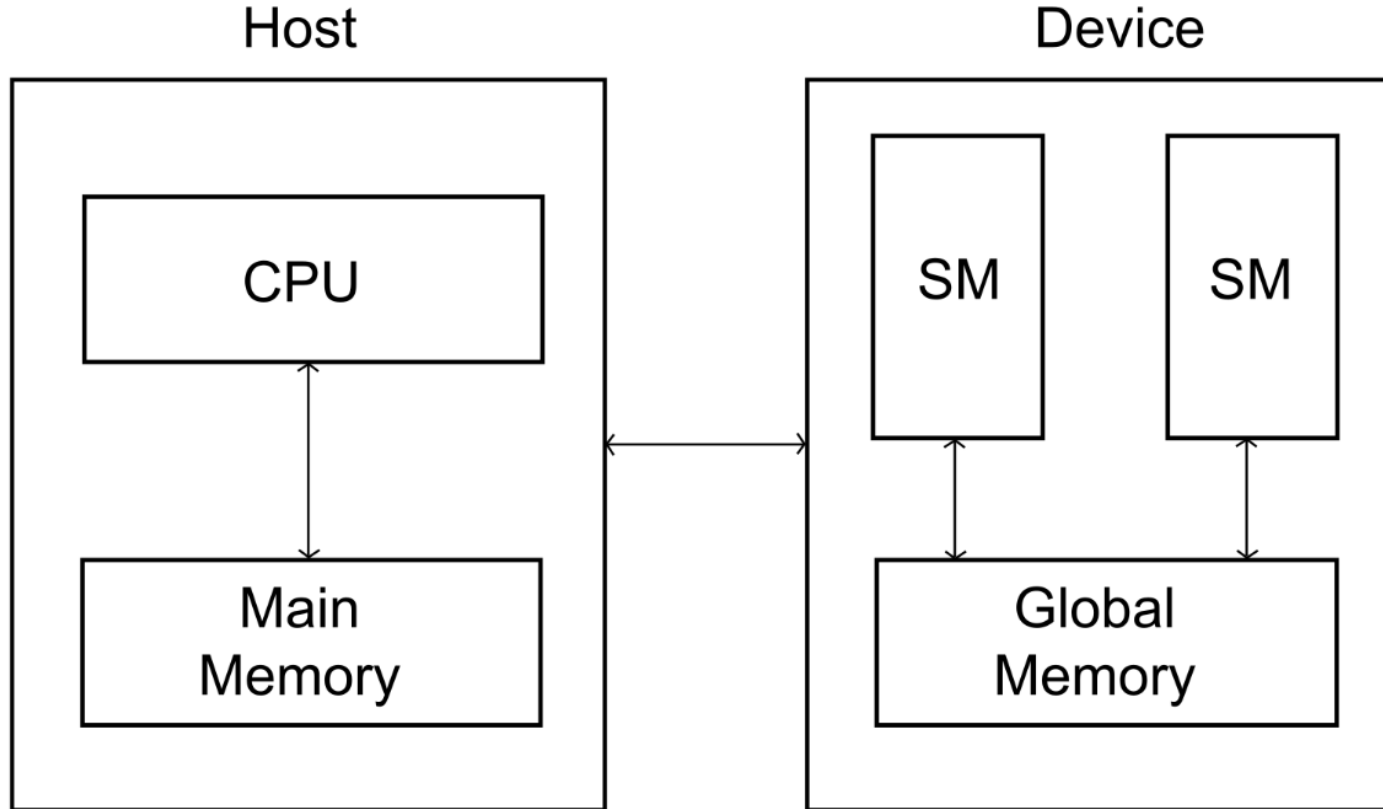
```
# x, y initialized here
```

```
for i in range(1_000_000):  
    if x[i] > 0:  
        y[i] = 2 * x[i]  
    else:  
        y[i] = -x[i]
```

**Worst case performance:
1/8 of peak performance
(Why??)**

Time
(clock ticks)

GPU & CPU



GPU vs CPU

- Clock frequency:
GPU has lower clock speed (5090 is ~ 2 GHz)
- Memory bandwidth:
GPU has much higher memory bandwidth
(4090: ~1 TB/s vs DDR5: ~50 GB/s)
- You should use CPU when
 - Complex logic (e.g. lots of branching)
 - Latency oriented (example?)
- You should use GPU when
 - SIMD-friendly and “simple” operations
 - Throughput oriented (example?)

Useful resources

- An Introduction to Parallel Programming by Peter Pacheco, Chapters 2 & 6
 - Covers architectures of CPU and GPU in more details
- Machine Code for Beginners (Usborne Computer Guide)
 - For those really interested in the details of how a CPU executes codes